

Чек-лист проверки потоков

Создана пользователем Леликов Александр Александрович. Последнее обновление: 23 мар., 2026 • Время чтения: 4 мин.

Оглавление:

- Введение
 - Назначение
 - Статус документа
 - Порядок применения
 - Ограничения
- Легенда критичности
- Чек-лист проверки потоков Apache NiFi

Введение

Настоящий чек-лист разработан в рамках **Рекомендаций по проектированию потоков Apache NiFi** и предназначен для оценки качества разработанных потоков обработки данных.

Назначение

Чек-лист выполняет следующие функции:

- **Самопроверка разработчика** — позволяет оценить соответствие потока принятым подходам перед передачей на тестирование или ревью
- **Унификация подхода** — обеспечивает единые критерии оценки для всех участников процесса
- **Снижение рисков** — помогает выявить потенциальные проблемы безопасности, производительности и сопровождаемости на ранних этапах

Статус документа

Документ имеет **рекомендательный характер**. Это означает, что:

- Проверки, зафиксированные в чек-листе, отражают накопленный опыт и лучшие практики, но не являются безусловным требованием.
- В случае противоречия между положениями чек-листа и требованиями Заказчика (техническое задание, архитектурные требования, политики безопасности, регламенты эксплуатации) **приоритет имеют требования Заказчика**.
- Отступление от рекомендаций чек-листа допускается при наличии обоснования (техническая необходимость, ограничения инфраструктуры, специфика бизнес-логики) и документальной фиксации принятого решения.

Порядок применения

Чек-лист применяется итеративно на следующих этапах жизненного цикла потока:

Этап	Участник	Объём проверки
Самопроверка	Разработчик	Все пункты (особое внимание — блокирующим и критическим)
Code Review	Tech Lead / Старший разработчик	Подтверждение выполнения всех пунктов
Приёмка в тестовой среде	Тестировщик / Интегратор	Проверки, связанные с тестируемостью, изоляцией сред, воспроизводимостью

Ограничения

Чек-лист не заменяет:

- Функциональное тестирование потока.
- Нагрузочное тестирование.
- Проверки, специфичные для конкретного домена (финансы, медицина, промышленные данные и т.д.).

Для таких случаев чек-лист может быть дополнен проектными или доменными проверками.

Легенда критичности

Уровень	Описание	Действие при нарушении
Блокирующий	Нарушение безопасности, потеря данных, невозможность эксплуатации	Приёмка невозможна до устранения
Критичный	Существенное нарушение стандартов, риски стабильности, нарушение SLA	Требуется исправление; может быть принято с планом исправления по согласованию
Рекомендательный	Рекомендация по улучшению сопровождаемости, не влияет на функциональность	Может быть принято с техническим долгом

Чек-лист проверки потоков Apache NiFi

№	Категория	Что проверяем	Критерий проверки (как должно быть)	Критичность
1. Структура и визуальное оформление				
1.1	Логическая структура	Поток построен сверху вниз, ветвления — слева направо	Схема читается как последовательный процесс; обратные связи не нарушают общей визуальной читаемости	Рекомендательный
1.2	Группировка	Крупные логические этапы выделены в Process Group	Корневой уровень содержит только стартовые процессоры и Process Group; одиночные процессоры (кроме источников) на корневом уровне отсутствуют	Рекомендательный
1.3	Размер Process Group	Process Group не перегружена	Если логический этап требует более 20–30 процессоров, рассмотреть декомпозицию; для < 3 процессоров отдельная PG не обязательна	Рекомендательный
1.4	Цветовая маркировка	Используется стандартная цветовая схема	Жёлтый (`#ffff00`) — взаимодействие с внешними системами; синий (`#ade8ff`) — процессоры условий; красный (`#e32222`) — запись ошибок	Рекомендательный
1.5	Комментарии	Process Group содержит описание	Заполнены комментарии для Process Group; для процессоров со сложной логикой (скрипты, нестандартные настройки) также добавлены пояснения; при необходимости используются Label	Рекомендательный
2. Именование				

№	Категория	Что проверяем	Критерий проверки (как должно быть)	Критичность
2.1	Префиксы	Используются обязательные префиксы	`f_` — Flow; `pg_` — Process Group; `PC_` — Parameter Context; `a_` — атрибуты; `t_` — тестовые атрибуты (удаляются после тестирования); `cs_` — Controller Services	Критичный
2.2	Процессоры	Название отражает действие	Формат: глагол + существительное (PascalCase). Пример: `ExtractJsonAttributes`, `ValidatePassportFormat`. Для процессоров с вкладкой Advanced указывается суффикс `[A]`	Рекомендательный
2.3	Порты	Названия портов единообразны	Примеры: `in_start`, `out_success`, `out_error`	Рекомендательный
2.4	Параметры контекста	Стиль именования	Пример: `DATABASE_HOST`, `INPUT_DIRECTORY_PATH`	Рекомендательный
2.5	Controller Services	Префикс и постфикс	Формат: `cs_Имя_постфикс`. Постфикс `_default` — для стандартных; `_avro` — для работающих с AVRO-схемами	Рекомендательный
2.6	Обязательные атрибуты	Присутствуют в начале потока	Объявлены: `a_developer`, `a_flow_folder`, `a_flow_name`, `a_id_run_flow`, `a_task_number`	Критичный
3. Атрибуты и данные				
3.1	Разделение content / attributes	Техническая информация не вынесена в content, бизнес-данные — не в attributes	Content содержит исходные и преобразованные бизнес данные; attributes — служебную информацию для маршрутизации, логирования, трассировки	Критичный
3.2	Область действия атрибутов	Атрибуты не передаются дальше, чем необходимо	Атрибуты используются только внутри логического этапа; временные и диагностические атрибуты не выходят за пределы PG	Рекомендательный
3.3	Удаление атрибутов	Атрибуты удаляются после использования	После использования атрибутов по назначению (кроме обязательных, п. 6.2.8) они удаляются	Рекомендательный
3.4	Запрет на большие атрибуты	Размер атрибутов ≤ 100 kB	Отсутствуют атрибуты с объемом более 100 kB;	Критичный
3.5	Чувствительные данные	Отсутствуют в открытом виде	Пароли, токены, ключи не хранятся в attributes, комментариях, описаниях; вынесены в Parameter Context с типом Sensitive	Блокирующий
4. Конфигурация и управление				
4.1	Parameter Context	Все переменные вынесены	В процессорах отсутствуют захардкоженные URL, пути, логины, пороги, таймауты; используются параметры контекста	Блокирующий

№	Категория	Что проверяем	Критерий проверки (как должно быть)	Критичность
4.2	Версионность	Поток привязан к NiFi Registry	Изменения фиксируются в реестре	Критичный
5. Обработка ошибок				
5.1	Логирование ошибок	Фиксируются обязательные поля	Для каждой ошибки зафиксированы: время, этап потока, идентификатор запуска (`a_id_run_flow`), краткое описание причины	Критичный
5.2	Временные сбои	Настроен механизм retry	Для внешних систем настроены повторные попытки с паузами	Критичный
6. Логирование и трассировка				
6.1	Логирование ключевых этапов	Логируются извлечение, запись, преобразования	Логируются: извлечение из источников, запись в целевые системы, агрегация, фильтрация, JOLT, скрипты	Критичный
6.2	Локальность логирования	Каждая Process Group логирует самостоятельно	Логирование выполняется внутри каждой PG для изоляции ответственности	Рекомендательный
6.3	Формат логирования	Присутствуют ключевые атрибуты	В логах фиксируются: идентификаторы источника, временные метки, количество записей.	Критичный
7. Производительность и устойчивость				
7.1	Back Pressure	Ограничения очередей изменены обоснованно, (если имеется)	Если лимиты Back Pressure отличаются от дефолтных (10000 файлов / 1 ГБ), то такое изменение обосновано (например, расчётной пропускной способностью источника) и задокументировано	Критичный
7.2	Concurrent Tasks	Настроено осознанно, (если имеется)	Для операций записи во внешние системы — минимальная параллельность; для вычислительных — повышенная.	Критичный
7.3	Размер FlowFile	Отсутствует необоснованное дробление данных	Данные не разбиты на множество мелких FlowFile без явного обоснования (требование независимой маршрутизации, разные SLA, ограничения внешней системы)	Критичный
7.4	Поведение при длительных сбоях	Очереди не используются как хранилище	При длительной недоступности целевой системы (срок определяется требованиями Заказчика / проектной документацией) данные, в очередях, не накапливаются бесконечно; срабатывают ограничения очередей, обработка приостанавливается	Критичный
8. Безопасность и доступ				

№	Категория	Что проверяем	Критерий проверки (как должно быть)	Критичность
8.1	Restricted Components	Использование обосновано	Если в потоке используется процессор категории Restricted Components (красный щит), то применение такого компонента обосновано: задача не решается стандартным процессором, либо альтернативы существуют, но менее оптимальны по производительности, сложности или сопровождаемости.	Критичный
8.2	Доступ к данным через Parameter Context	Секреты не передаются через код	Чувствительные данные доступны только через Parameter Context	Блокирующий
9. Тестируемость и сопровождаемость				
9.1	Тестовые атрибуты	Удалены перед релизом	Атрибуты с префиксом `t_` отсутствуют в продуктивном потоке	Критичный
9.2	Воспроизводимость	Поток может быть развёрнут в другой среде	Используются Parameter Context для окружений	Блокирующий

Нет меток